

SYSTEM ANALYSIS AND DESIGN

Complete Model Questions – Exam-Oriented Answers

BIT 123 | Year 2 / Semester 4
Model Sets 1–4
Definitions • Proper Examples • Detailed Answers • Seven-point Differences

Contents

Model Set 1	
Model Set 2	
Model Set 3	
Model Set 4	
Quick Revision	
The source paper uses Section A = 2 marks, Section B = 7 marks, and Section C = 15 marks.	 filecite☆turn1 file0☆L23-L27↕↔

MODEL SET 1

The uploaded paper defines Section A as 2 marks, Section B as 7 marks, and Section C as 15 marks.	 filecite☆turn1 file0☆L28-L55↕↔
---	--

Section A – Very Short Answer Questions

1. Define System Analysis and mention any one primary objective of performing analysis in SDLC.

System Analysis is the systematic study of an existing or proposed information system to understand its processes, data, problems, requirements, and interactions. A primary objective is to identify and document user and business requirements accurately.
2. State any two major characteristics of an Information System.

An information system collects, processes, stores, and distributes information. It also supports organizational operations, management, control, or decision-making.
3. Define Data Flow Diagram (DFD) and state its primary role in structured analysis.

A DFD is a graphical representation of how data moves between external entities, processes, and data stores. Its primary role is to model system processes and data movement clearly.
4. Mention any two standard graphical symbols used in an Entity Relationship Diagram (ERD).

A rectangle commonly represents an entity and an oval represents an attribute. A diamond is commonly used for a relationship.
5. What is a Data Dictionary? State two entries commonly cataloged within it.

A Data Dictionary is a centralized repository of metadata describing system data. Two common entries are data elements and data stores.
6. Define Decision Table and identify its four essential quadrant components.

A Decision Table is a tabular method for representing complex business rules. Its four components are condition stubs, condition entries, action stubs, and action entries.
7. State any two distinct types of feasibility studies conducted during system planning.

Technical feasibility and economic feasibility are two types. Operational and schedule feasibility are other common types.
8. Define Logical Design and distinguish it briefly from Physical Design.

Logical Design describes what data, processes, inputs, and outputs the system needs without committing to specific technology. Physical Design specifies how those requirements will be implemented using actual databases, hardware, software, networks, and interfaces.
9. Define coupling and cohesion in modular system design.

Coupling is the degree of dependency between modules. Cohesion is the degree to which responsibilities within one module belong together. Good design generally aims for loose coupling and high cohesion.

10. Define system testing and state any two types of software maintenance.

System testing tests the complete integrated system against its requirements. Two maintenance types are corrective and adaptive maintenance; perfective and preventive are also common.

Section B – Short Answer Questions

11. Explain the role of System Analysis and discuss the core technical, analytical, and interpersonal responsibilities of a System Analyst.

System Analysis bridges organizational needs and technical implementation. A system analyst studies the current system, identifies problems, gathers requirements, models processes and data, evaluates alternatives, and communicates the proposed solution.

Area	Responsibilities
Technical	Understand software, databases, networks, interfaces, security and modeling tools; prepare technical models/specifications.
Analytical	Study processes, identify problems, analyze requirements, evaluate alternatives, assess feasibility and risk.
Interpersonal	Interview users, conduct meetings, negotiate conflicts, communicate with management/developers and manage expectations.

Example: For a college system, the analyst interviews admission and accounts staff, models their processes, identifies duplicate records, and produces DFD/ERD and requirement specifications.

12. Explain the Context Diagram (Level 0 DFD). Discuss significance and rules of decomposing it into Level 1.

A Context Diagram represents the entire system as one process and shows its boundary and interactions with external entities. It emphasizes major inputs and outputs.

- Use one process for the whole system.
- Show external entities outside the system.
- Name every data flow.
- Normally omit internal data stores at context level.
- Level 1 decomposes the single process into major subprocesses.
- Preserve the parent diagram's external inputs/outputs.
- Maintain balancing between parent and child diagrams.

Example: Student Information System: Student → admission details → [SIMS] → confirmation → Student; Accounts → fee data → [SIMS] → reconciliation report → Accounts.

Level 1 may contain Admission Processing, Fee Processing, and Reporting, while retaining balanced external flows.

13. Discuss Data and Process Modeling tools. Explain CASE tools.

Data and process modeling tools represent system data, processes, relationships, and movement. Examples include DFDs, ERDs, decision tables, decision trees, structure charts, and UML diagrams.

CASE feature	Advantage
Repository	Centralizes models, requirements, definitions and metadata.
Diagram generation	Creates and maintains DFD, ERD and UML diagrams.
Consistency checking	Helps detect inconsistent names and model elements.
Code skeleton generation	Some tools generate initial code/framework structures.
Traceability	Links requirements, models, design, code and tests.
Productivity	Automates repetitive modeling/documentation work.
Maintenance	Central repository makes change management easier.

14. Explain Decision Trees and Structured English. Compare them with Decision Tables for complex business logic.

A Decision Tree represents conditions as branches and actions as leaves. It is useful when decisions follow a visible sequence. Structured English uses controlled English with IF-THEN-ELSE, CASE, WHILE, and similar constructs to describe procedural logic.

Technique	Suitable when	Main strength
Decision Tree	Decisions follow clear sequential branches.	Visual and easy to trace.
Structured English	Logic is procedural, nested, or step-oriented.	Readable and close to program logic.
Decision Table	Many combinations of conditions affect actions.	Systematically covers rule combinations.

Example: A loan decision with income, collateral, and credit status can be represented as a tree; many independent condition combinations are often easier to verify in a decision table.

15. Discuss the Feasibility Study process and Cost/Benefit Analysis.

A feasibility study evaluates whether a proposed system is practical and worthwhile. It examines technical, operational, economic, and schedule factors.

1. Define scope, objectives and alternatives.

- 2. Assess technical resources and skills.
- 3. Assess operational fit and user acceptance.
- 4. Estimate development, training, operation and maintenance costs.
- 5. Identify tangible and intangible benefits.
- 6. Assess schedule constraints.
- 7. Compare alternatives and document findings.

Category	Meaning	Example
Direct cost	Directly attributable to the project.	Developer salary assigned to the project.
Indirect cost	Supporting/overhead cost.	Shared administration or infrastructure.
Tangible benefit	Measurable financially.	Reduced paper/clerical expense.
Intangible benefit	Difficult to assign exact monetary value. Improved customer satisfaction.	

Cost/benefit analysis supports investment decisions by comparing expected costs and benefits over time and under stated assumptions.

16. Explain Structured System Design, modular design, functional strength, structure charts, and minimizing coupling.

Structured System Design converts analyzed processes into a hierarchy of manageable modules. Modular design divides the system into focused components. Functional strength means each module should have a clear, related responsibility. A structure chart shows the hierarchy and module calls.

- Use focused modules with high cohesion.
- Pass only required data between modules.
- Prefer data parameters over global/shared data.
- Use information hiding to protect internals.
- Avoid unnecessary control flags.
- Define clear interfaces.

Example: A billing system can contain Customer Management, Invoice Calculation, Payment Processing, and Report Generation modules.

Section C – Long Answer Questions

17. COMPULSORY CASE STUDY – Academic Institution Automation

The scenario describes replacing manual admission, enrollment, fee, grade, and library records with SIMS to reduce duplication, delayed results, fee inconsistency, and misplaced records.

a. Stakeholders

Stakeholder	Interaction
Students	Submit admission/enrollment data; view fees, results and permitted records.
Admissions staff	Verify applications, manage admission and enrollment.
Accounts staff	Record fees, issue receipts, reconcile payments.
Faculty	Enter grades and access course/student information.
Library staff	Manage issue/return records.
Management	View reports and monitor performance.
IT/System administrator	Users, security, backups, configuration and support.

b. Fact-finding techniques

Technique	Application	Advantage
Interview	Admission, accounts, faculty, library and management.	Detailed information and follow-up.
Questionnaire	Students/staff at larger scale.	Fast and standardized.
Observation	Watch real admission/fee/library workflows.	Finds undocumented practices.
Document sampling	Review ledgers, forms, receipts and grade sheets.	Reveals current fields and formats.

c. Context and Level 1 DFD

Context: STUDENT → admission/enrollment/fee details → [SIMS] → confirmation/results/fee status → STUDENT

ADMISSIONS STAFF → application verification → [SIMS] → admission status → ADMISSIONS STAFF

ACCOUNTS → payment/fee data → [SIMS] → receipts/reconciliation reports → ACCOUNTS

MANAGEMENT ← reports ← [SIMS]

Level 1: STUDENT → (1.0 Admission Processing) ↔ D1 Student Records; (1.0) → confirmation → STUDENT.

STUDENT/ACCOUNTS → (2.0 Fee Processing) ↔ D2 Fee Records; (2.0) → receipt/status → STUDENT; (2.0) → reconciliation → ACCOUNTS.

d. ERD

Entity	Key attributes	Relationships
STUDENT	Student_ID PK, Name, DOB, Address, Phone, Email	1:M ENROLLMENT; 1:M FEE; 1:M GRADE; 1:M LIBRARY_RECORD
COURSE	Course_ID PK, Course_Name, Credit_Hours	1:M ENROLLMENT; 1:M GRADE
ENROLLMENT	Enrollment_ID PK, Student_ID FK, Course_ID FK, Semester	M:1 STUDENT; M:1 COURSE
FEE	Fee_ID PK, Student_ID FK, Amount, Due_Date, Status	M:1 STUDENT; 1:M PAYMENT
PAYMENT	Payment_ID PK, Fee_ID FK, Amount, Payment_Date, Method	M:1 FEE
BOOK	Book_ID PK, Title, Author, ISBN	1:M LIBRARY_RECORD

The ERD should show entities as rectangles, attributes, primary keys, foreign keys, and relationship cardinalities such as 1:M.

18. Critically analyze SDLC. Compare Waterfall, Prototyping, Spiral and Incremental models for an enterprise online banking portal.

SDLC is a structured framework for planning, analyzing, designing, developing, testing, implementing and maintaining systems.

Basis	Waterfall	Prototyping	Spiral	Incremental
Risk	Upfront planning; late changes can be costly.	Prototype exposes requirement/usability risks early.	Explicit risk analysis in repeated cycles.	Risk distributed across smaller releases.
Cost	Planned by phase; changes may increase cost.	Prototype adds early cost but can reduce misunderstanding.	Can be costly due to repeated risk analysis.	Staged investment.
Requirements	Less flexible after baseline.	Highly flexible during refinement.	Flexible between cycles.	Flexible between increments.
Delivery	Usually one major release.	Prototype may evolve or be discarded.	Successive risk-driven cycles.	Successive usable releases.
User involvement	Mostly requirements/reviews.	High user feedback.	Review at cycle milestones.	Feedback after increments.
Best fit	Stable requirements.	Unclear/UI-heavy requirements.	Large/high-risk systems.	Systems divisible into releases.
Banking relevance	Strong governance but less flexible.	Useful for UI discovery.	Useful for security/technology risk analysis.	Useful for controlled feature rollout.

For an enterprise banking portal, the development approach should prioritize security, compliance, auditability, rigorous testing, and controlled change. A risk-driven iterative approach such as Spiral can be appropriate where risks are substantial; incremental delivery can also be combined with strong governance for controlled releases.

19. Discuss OOAD using UML and explain Use Case, Class, Sequence and State Transition Diagrams.

OOAD models systems as interacting objects/classes that combine data and behavior. UML provides standardized notation.

Diagram	Purpose / notation
Use Case	Functional requirements; actors, use cases, boundary, associations, «include», «extend».
Class	Structural model; classes, attributes, operations, associations, aggregation/composition, generalization.
Sequence	Behavior over time; lifelines, messages and activation bars.
State Transition	Object states and event-triggered transitions.

Example: Online shopping checkout: Customer → Cart: checkout(); Cart → Order: createOrder(); Order → PaymentService: authorizePayment(); Order → Inventory: reserveItems(); Order → Customer: confirmation.

20. Explain System Implementation, installation strategies, SQA mechanisms and four maintenance types.

Implementation prepares the system for operational use through coding completion, testing, data migration, installation, training, documentation, conversion and support.

Strategy	Description	Risk/resource characteristic
Direct	Immediate switch from old to new.	Fast; higher operational risk.
Parallel	Old and new run together.	Lower risk; higher resource overhead.
Pilot	Introduce to one selected group/site.	Risk contained to pilot.
Phased	Introduce modules/stages gradually.	Controlled transition.

SQA mechanism	Meaning
Formal Technical Review	Structured review of requirements/design/code or other work products.
Walkthrough	Author presents work to reviewers for feedback and defect discovery.
Inspection	Highly formal, checklist-driven defect-detection review with defined roles and records.

Maintenance Purpose	Example
Corrective	Fix discovered faults. Correct wrong fee calculation.
Adaptive	Adapt to changed environment. Support new OS/regulation.
Perfective	Improve functionality/usability/performance. Add better search.
Preventive	Reduce future faults and improve maintainability. Refactor risky code and improve tests.

MODEL SET 2

The uploaded source lists Model Set 2 in Section A through Section C. filecite turn1 file0 L115-L150 

Section A – Very Short Answer Questions

1. Differentiate between TPS and MIS.

TPS records routine day-to-day transactions. MIS summarizes processed organizational data into reports useful for management monitoring and decisions.

2. List any four key stakeholders involved in enterprise software development.

Users/customers, managers/sponsors, system analysts, developers, testers, IT administrators, and regulators are examples; any four are sufficient.

3. What is an external entity in a DFD? How is it symbolized?

An external entity is a person, organization, or external system that exchanges data with the system but lies outside its boundary. It is commonly shown as a rectangle.

4. Define cardinality and modality in ER modeling.

Cardinality describes the maximum number of related instances, such as 1:1, 1:M, M:N. Modality describes minimum participation, commonly optional (0) or mandatory (1).

5. Explain Data Store repository entries in a Data Dictionary.

They describe the data store's name, purpose, contents, source, users/processes, structure and other metadata.

6. When is Structured English preferred over a Decision Table?

When logic is procedural, sequential, nested, or easier to explain as steps than as many combinations of conditions.

7. What is a Use Case Actor? Give one example.

An actor is an external role interacting with a system. Example: Customer in an online shopping system.

8. Define Payback Period.

The Payback Period is the time needed for cumulative cash inflows or savings to recover the initial project investment.

9. State two objectives of effective UI and input form design.

Make interaction clear and easy; reduce input errors while maintaining accurate, consistent data.

10. Distinguish Alpha and Beta Testing.

Alpha testing is generally internal and controlled. Beta testing is conducted by selected external/real users in more realistic conditions before final release.

Section B – Short Answer Questions

11. Examine the evolving role of a System Analyst.

The modern analyst combines business analysis, requirements engineering, process improvement, data analysis, technology evaluation, stakeholder facilitation and coordination between business and technical teams.

- Translate business goals into requirements.
- Model processes, data and interactions.
- Assess feasibility, risks and constraints.
- Facilitate communication and resolve requirement conflicts.
- Support change management and validation.
- Help align digital solutions with organizational objectives.

Example: In online banking, the analyst translates customer and banking requirements into specifications developers and testers can implement and verify.

12. Explain DFD rules and Black Hole, Miracle and Gray Hole errors.

DFDs should have meaningful process names, named data flows, valid connections, and balanced decomposition. A process should have meaningful inputs and outputs.

Error	Meaning	Example
Black Hole	Input but no output.	Fee calculation receives data but produces no result.
Miracle	Output without input.	Report appears without any source data.

Gray Hole Inputs exist but cannot reasonably produce stated outputs. Student_ID alone produces a complete financial report without access to required data.

13. Discuss Data Dictionaries, data structures, elements and stores.

A Data Dictionary formally defines system data and promotes consistency across analysis, design, database and interface artifacts.

Entry	Typical information
Data element	Name, meaning, type, length, allowed values, validation.
Data structure	Group of related elements, e.g., Student = ID + Name + Address.
Data store	Name, purpose, contents, source and processes accessing it.
Data flow	Name, source, destination and structure.

14. Construct a Decision Table for the retail discount policy.

Conditions / Rules	R1	R2	R3	R4
Purchase > \$500?	Y	Y	N	N
Member?	Y	N	Y	N
Discount	15%	10%	5%	0%

The table covers all four possible combinations of the two binary conditions.

15. Explain requirement gathering including JAD and rapid questionnaires.

Technique	Merit	Limitation
Interview	Deep, flexible information.	Time-consuming.
Questionnaire	Fast collection from many people.	Limited depth.
Observation	Shows actual workflow.	Can require substantial time.
JAD	Fast group consensus and shared understanding.	Needs skilled facilitation.
Rapid questionnaire	Quick focused feedback.	Less detail than interviews/JAD.

Combining techniques usually gives better coverage than relying on one method.

16. Discuss Logical Design versus Physical Design.

Seven-point difference: Logical vs Physical Design

Basis	Logical Design	Physical Design
1. Focus	What the system needs.	How it is implemented.
2. Technology	Mostly independent.	Technology dependent.
3. Database	Entities, relationships, normalization.	Tables, indexes, storage and DBMS settings.
4. I/O	Logical content and flow.	Actual screens, forms, files and reports.
5. Performance	Logical requirements.	Physical optimization and resource use.
6. Stability	More stable across technologies.	Can change with implementation choices.
7. Example	Student has ID, Name and Course relationship.	SQL tables, indexes, server and storage.

Section C – Long Answer Questions

17. COMPULSORY CASE STUDY – Hospital Information & Appointment System

The source case requires patient registration, doctor schedule search, online appointments, EMRs, e-prescriptions, lab report uploads, billing and insurance claims.

filecite turnl file0 L167-L179

a. External entities and requirements

Entity	Interaction
Patient	Register, search, book appointments and view permitted information.
Doctor	View schedules, update EMRs, issue prescriptions.
Laboratory	Upload investigation reports.
Accounts	Manage bills and insurance claims.
Insurance provider	Process/verify claims.
Administrator	Manage users, security and system configuration.

Functional requirements: registration; schedule search and appointment booking; EMR and e-prescription; lab uploads; billing and claims.

Non-functional requirements: security/access control; reliability/availability; acceptable response time; auditability of clinical and financial changes.

b. Context Diagram

PATIENT ↔ [HIAS] ↔ DOCTOR

LAB → lab reports → [HIAS] → report status → LAB

ACCOUNTS ↔ [HIAS] ↔ INSURER

c. Level 1 DFD

PATIENT → (1.0 Register/Book Appointment) ↔ D1 Patient Data

(1.0) ↔ D2 Doctor Schedule → appointment confirmation → PATIENT

DOCTOR → (2.0 Manage EMR/Prescription) ↔ D3 EMR → e-prescription → PATIENT

d. Feasibility

Type	Assessment
Technical	Feasible with secure infrastructure, database, backups, access control and integration.
Operational	Requires workflow redesign, training, acceptance and clinical safety procedures.
Economic	Costs include development, migration, training and support; benefits include less paperwork and faster scheduling.
Schedule	Use staged delivery, prioritizing registration/appointments/core records before advanced integrations if deadlines are tight.

18. Explain software architecture modularity, cohesion and coupling.

Modularity divides a system into manageable components with defined interfaces. High cohesion keeps related responsibilities together; loose coupling minimizes dependencies between modules.

Cohesion level	Meaning
Coincidental	Unrelated tasks grouped together; weakest.
Logical	Related by category but selected through control.
Temporal	Tasks grouped because they occur at the same time.
Procedural	Tasks grouped because they follow a procedure.
Communicational	Operations use the same data.
Sequential	Output of one part becomes input to another.
Functional	All parts contribute to one well-defined function; strongest.

Coupling level	Meaning
Data	Only required data parameters passed; generally loose.
Stamp	Whole structure passed though only part is needed.
Control	Control information changes another module's behavior.
External	Dependency on external formats/protocols/devices.
Common	Modules share global/common data.
Content	One module accesses another's internals; tightest.

High cohesion makes modules easier to understand and change. Loose coupling reduces the chance that a change or fault in one module spreads to others.

19. Discuss UML and provide Class and Sequence diagrams for online shopping checkout.

Structural models describe system elements and relationships; behavioral models describe actions and interactions.

Model	Elements
Class Diagram	Classes, attributes, operations and relationships.
Sequence Diagram	Lifelines, messages, activation bars and time order.

Class diagram: Customer 1—1 Cart; Cart 1—* CartItem; CartItem *—1 Product; Customer 1—* Order; Order 1—* OrderItem; OrderItem *—1 Product; Order → PaymentService.

Sequence: Customer → Cart: checkout(); Cart → Order: createOrder(); Order → PaymentService: authorizePayment(); PaymentService → Order: approved; Order → Inventory: reserveItems(); Order → Customer: confirmation.

20. Define System Testing. Differentiate Unit, Integration, System and Acceptance Testing; explain regression and FTR.

System Testing evaluates the complete integrated system against specified requirements.

Seven-point difference: Unit vs Integration

Basis	Unit Testing	Integration Testing
1. Scope	Individual module/unit.	Interactions among modules.
2. Aim	Verify local logic.	Verify interfaces/interactions.
3. Typical performer	Developer.	Developer/test team.
4. Dependencies	Often isolated.	Increasingly integrated.
5. Example	Test calculateFee().	Test admission + fee module.
6. Timing	Early and continuously.	After integration.
7. Defects	Component logic errors.	Interface/data communication defects.

Seven-point difference: System vs Acceptance

Basis	System Testing	Acceptance Testing (UAT)
1. Scope	Complete integrated system.	Business/user acceptance.
2. Aim	Verify system requirements.	Verify business needs/acceptance criteria.
3. Performer	QA/test team.	Users/business representatives.
4. Environment	Controlled test environment.	Representative acceptance environment.
5. Example	Full checkout workflow test.	Business validates approved checkout process.
6. Basis	System specifications.	Acceptance criteria/business requirements.
7. Outcome	Release readiness/defects.	Acceptance decision or required changes.

Regression testing repeats relevant tests after changes to detect unintended breakage. Formal Technical Reviews examine work products such as requirements, design and code early to detect defects and ensure standards.

MODEL SET 3

The source lists Model Set 3 questions 1–20. turn1 file0 L209-L243 

Section A – Very Short Answer Questions

1. State two main limitations of the traditional Waterfall Model.

Requirements are expected to be defined early, making later changes costly; user feedback and working software generally appear relatively late.

2. What is an Information System boundary and environment?

The boundary defines what is inside the system. The environment consists of external people, organizations, systems and conditions interacting with or influencing it.

3. Differentiate functional and non-functional requirements.

Functional requirements specify what the system does. Non-functional requirements specify quality attributes or constraints such as performance, security and usability.

4. What is a recursive relationship in an ERD?

A relationship in which an entity relates to another instance of the same entity. Example: Employee supervises Employee.

5. Define Upper-CASE vs Lower-CASE tools.

Upper-CASE supports early planning, requirements and analysis/modeling. Lower-CASE supports later design, coding, testing and maintenance.

6. Purpose of an Extended Entry Decision Table.

It models complex rules where conditions may have multiple values/ranges rather than only simple yes/no entries.

7. Purpose of a State Machine Diagram.

It shows states of an object/system and transitions caused by events, conditions or actions.

8. What is ROI and how is it expressed?

ROI measures return relative to investment. $ROI = (Net\ Benefit / Investment\ Cost) \times 100\%$.

9. What is a Structure Chart and what do arrows with open/solid circles indicate?

A Structure Chart shows hierarchical modules and module calls. Its arrows show communication/control or data passed between modules; exact open/solid-circle meaning depends on the notation convention used.

10. Define preventive software maintenance.

Section B – Short Answer Questions

11. Explain major SDLC phases and primary deliverables.

Phase	Activities	Deliverable
Planning	Scope, objectives, feasibility.	Project plan/feasibility report.
Analysis	Requirements, process/data modeling.	Requirements specification, DFD/ERD.
Design	Architecture, database, UI, modules.	Design specification/schema/UI.
Development	Coding/configuration.	Source code/build.
Testing	Unit, integration, system, acceptance.	Test results/defect records.
Implementation	Migration, installation, training.	Operational system and deployment documents.
Maintenance	Correct/adapt/improve/prevent.	Updated releases and maintenance records.

12. Explain ERD drawing procedure, entity types, weak entities, attributes and key constraints.

1. Identify entities from requirements.
2. List attributes.
3. Select primary keys.
4. Identify relationships.
5. Determine cardinality and modality.
6. Resolve M:N relationships with associative entities.
7. Review the model for consistency.

Concept	Meaning	Example
Strong entity	Independently identifiable.	STUDENT(Student_ID, Name).
Weak entity	Depends on owner for identification/existence.	DEPENDENT using Employee_ID + Dependent_Name.
Attribute	Property of entity/relationship.	Student_Name.
Primary key	Uniquely identifies instance.	Student_ID.
Foreign key	References another key.	Course_ID in Enrollment.
Cardinality	Maximum related instances.	1:M.
Key constraint	Uniqueness/identification requirement.	Student_ID unique.

13. Describe Project Repository and how Data Dictionaries enforce consistency.

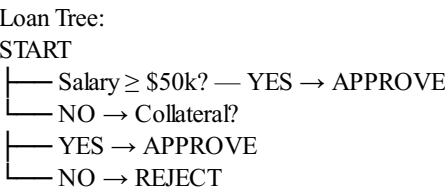
A Project Repository centrally stores requirements, diagrams, definitions, design artifacts, standards, versions and traceability information. A Data Dictionary provides authoritative definitions for data elements, structures, stores and flows, ensuring the same name, meaning and format are used in DFDs, ERDs, forms, reports and databases.

Example: Student_ID can be defined once as a unique required identifier and then reused consistently across admission, enrollment and fee models.

14. Differentiate Decision Tables and Decision Trees and draw a loan eligibility tree.

Seven-point difference: Decision Table vs Decision Tree

Basis	Decision Table	Decision Tree
1. Form	Table.	Branches/tree.
2. Best for	Many condition combinations.	Sequential decisions.
3. Coverage	Explicit combinations.	Paths represent combinations.
4. Readability	Systematic rule checking.	Visual path tracing.
5. Complexity	Can become wide.	Can become deep.
6. Missing rules	Easier to identify missing combinations.	May be missed if poorly designed.
7. Example	Discount policy.	Loan eligibility.



15. Explain Tangible vs Intangible Costs and Benefits with examples.

Category	Meaning	Example 1	Example 2
Tangible cost	Directly measurable financially.	Software license.	Server purchase.

Intangible cost	Difficult to value precisely.	Employee stress during transition.	Temporary loss of confidence.
Tangible benefit	Financially measurable.	Reduced printing cost.	Lower overtime.
Intangible benefit	Difficult to value precisely.	Higher satisfaction.	Better reputation.

16. Discuss Input and Form Design techniques.

- Use clear labels and logical grouping.
- Use appropriate controls such as lists, radio buttons and date pickers.
- Validate type, format, range, required fields and cross-field consistency.
- Minimize unnecessary fields and repeated entry.
- Use controlled values where possible.
- Give clear error messages and preserve valid entered data.
- Maintain consistent navigation and accessibility.

Example: A date-of-birth field should validate date format and a reasonable range; a department field can use a controlled list instead of free text.

Section C – Long Answer Questions

17. COMPULSORY CASE STUDY – E-Commerce Retail & Inventory Management

The source scenario requires accounts, catalog browsing, secure payment gateways, real-time inventory, automatic purchase-order drafting, dispatch, tracking and RMA processing.

a. External entities and Context Diagram

Entity	Interaction
Customer	Account, browsing, orders, payment status, returns.
Payment Gateway	Payment authorization/result.
Supplier	Purchase orders and confirmations.
Logistics/Carrier	Dispatch and tracking.
Warehouse	Stock and fulfillment information.
Management	Reports.

CUSTOMER ↔ [E-COMMERCE & INVENTORY PLATFORM] ↔ PAYMENT GATEWAY
[PLATFORM] ↔ SUPPLIER; [PLATFORM] ↔ LOGISTICS/CARRIER; [PLATFORM] ↔ WAREHOUSE; [PLATFORM] → MANAGEMENT.

b. Level 1 DFD

1.0 Order Processing: Customer → order → (1.0) → D1 Orders; (1.0) → payment request → 2.0.
2.0 Payment Verification: 2.0 ↔ Payment Gateway; 2.0 → status → 1.0.
1.0 → stock request → 3.0 Inventory Replenishment ↔ D2 Product_Inventory; reorder check → Purchase Order Draft → Supplier.

c. ERD

Entity	Attributes	Cardinality
Customer	Customer_ID PK, Name, Email, Address	1:M Order
Order	Order_ID PK, OrderDate, Status, Customer_ID FK	1:M OrderItem; M:1 Customer
Product	Product_ID PK, Name, Price, ReorderPoint	1:M OrderItem
Supplier	Supplier_ID PK, Name, Contact	1:M PurchaseOrder
Shipment	Shipment_ID PK, Order_ID FK, Carrier, TrackingNo, Status	Order 1:M/1:1 depending on policy
OrderItem	OrderItem_ID PK, Order_ID FK, Product_ID FK, Qty, UnitPrice	M:1 Order; M:1 Product
PurchaseOrder	PO_ID PK, Supplier_ID FK, Date, Status	M:1 Supplier

d. Data Dictionary – D1 Product_Inventory

Entry	Definition
Product_ID	Product identifier; references Product.
Warehouse_ID	Warehouse/location identifier.
Quantity_On_Hand	Current stock quantity.
Reserved_Quantity	Stock reserved for confirmed orders.
Reorder_Point	Threshold that triggers replenishment.
Reorder_Quantity	Suggested quantity to order.

18. Discuss Project Scheduling, Feasibility, Gantt, PERT/CPM and Critical Path.

Project scheduling identifies activities, dependencies, durations, resources, milestones and deadlines. Gantt charts show activities against calendar time and help track progress. PERT handles activity-time uncertainty using estimates; CPM identifies the longest dependency path.

Example: If Path A-B-C = $4+5+3 = 12$ days and A-D-C = $4+2+3 = 9$ days, A-B-C is the critical path in this simplified network. A delay in a critical activity can delay project completion.

19. Explain database design and transition from unnormalized data to 1NF, 2NF and 3NF.

Database design organizes data for integrity, efficient access and maintainability. Normalization reduces redundancy and update anomalies.

Unnormalized: ORDER(Order_ID, Customer_Name, Product1, Qty1, Product2, Qty2, ...). Repeating groups are present.

1NF: ORDER_LINE(Order_ID, Customer_ID, Customer_Name, Product_ID, Product_Name, Qty) with atomic values.

2NF: Separate partial dependencies: ORDER(Order_ID, Customer_ID, Customer_Name); PRODUCT(Product_ID, Product_Name); ORDER_LINE(Order_ID, Product_ID, Qty).

3NF: Move transitive customer details to CUSTOMER: CUSTOMER(Customer_ID, Customer_Name, Customer_Address); ORDER(Order_ID, Customer_ID); PRODUCT(Product_ID, Product_Name); ORDER_LINE(Order_ID, Product_ID, Qty).

Normalization reduces duplication and insert/update/delete anomalies and improves enterprise data consistency.

20. Explain System Conversion and Installation; compare Direct, Parallel, Pilot and Phased; discuss post-implementation review.

Seven-point difference: Direct vs Parallel

Basis	Direct Cutover	Parallel Operation
1. Changeover	Immediate switch.	Both systems run.
2. Risk	High.	Lower.
3. Resource	Lower after switch.	High.
4. Recovery	Rollback may be difficult.	Old system is fallback.
5. Speed	Fast.	Slow.
6. Suitable	Lower-risk/simple systems.	Critical systems.
7. Example	Small internal app.	Payroll/banking support.

Seven-point difference: Pilot vs Phased

Basis	Pilot Operation	Phased Rollout
1. Scope	One site/group first.	Modules/stages gradually.
2. Risk	Contained to pilot.	Contained by phase.
3. Learning	Feedback from pilot.	Feedback each phase.
4. Resources	Pilot support required.	Resources spread across stages.
5. Recovery	Stop pilot while others continue.	Delay/rollback affected phase.
6. Suitable	Multi-site organizations.	Large modular systems.
7. Example	One branch first.	Admissions, then fees, then reports.

Post-implementation review compares actual results with objectives, checks performance/security/user satisfaction, reviews costs/benefits/defects, documents lessons learned, and plans corrective actions and enhancements.

MODEL SET 4

The source lists Model Set 4 questions 1–20. filecite☆turn1 file0☆L300-L333↺

Section A – Very Short Answer Questions

1. What is an ESS and who are its primary users?

An Executive Support System presents summarized strategic information for senior-level planning and decisions. Its primary users are executives and senior managers.

2. Explain Data Flow Splitting and Merging in DFDs.

Splitting sends one data flow to multiple destinations/processes. Merging combines multiple flows into one when their data meaning and model justify the combination.

3. What is a composite attribute? Give an example.

An attribute that can be divided into meaningful sub-attributes. Example: Address = Street + City + Province + Postal_Code.

4. What is metadata and how is it documented in a Data Dictionary?

Metadata is data about data. A Data Dictionary records name, definition, type, size, allowed values, source, destination and relationships.

5. Meaning of Indifference Condition in a Decision Table.

A condition whose value does not affect the action for that rule; often represented by a dash (-).

6. Define generalization and specialization in UML Class Diagrams.

Generalization models a superclass and subclasses through inheritance. Specialization creates a more specific class from a general class.

7. What is Operational Feasibility and why is user acceptance central?

It assesses whether a proposed system will work effectively in real operations. User acceptance matters because users must be able and willing to adopt it.

8. Define stamp and control coupling.

Stamp coupling passes a whole data structure when only part is needed. Control coupling passes information that influences another module's behavior.

9. What is a Structure Chart Module Call?

It is a relationship showing that one module invokes another module to perform a task.

10. Distinguish Adaptive and Perfective Maintenance.

Adaptive maintenance changes software for a changed environment; perfective maintenance improves functionality, usability or performance according to evolving needs.

Section B – Short Answer Questions

11. Compare Structured Analysis & Design (SAD) with Object-Oriented Analysis & Design (OOAD).

Seven-point difference: SAD vs OOAD

Basis	SAD	OOAD
1. Paradigm	Functions/processes and data flows.	Objects/classes combining data and behavior.
2. Core models	DFD, ERD, structure chart.	UML use case, class, sequence, state.
3. Decomposition	Functional.	Objects/classes.
4. Data/behavior	Often modeled separately.	Encapsulated together.
5. Reuse	Procedural modules/components.	Classes, composition, inheritance/interfaces.
6. Best fit	Process-centric systems.	Complex interacting entities.
7. Example	Payroll processes/modules.	Employee, PaySlip and TaxCalculator objects.

12. Explain DFD Balancing with an illustrative diagram.

DFD balancing means external inputs and outputs of a parent process remain consistent with the child diagram.

Level 0: CUSTOMER → Order → [1.0 Order System] → Confirmation → CUSTOMER

Level 1: CUSTOMER → Order → (1.1 Capture) → (1.2 Validate) → (1.3 Confirm) → Confirmation → CUSTOMER

- Do not add unexplained boundary flows.
- Do not omit parent boundary flows.
- Child may introduce internal processes/data stores.
- Child should explain how parent inputs become parent outputs.

13. Discuss CASE tools in repository management, diagram generation, code skeleton generation and traceability.

Area	Role
Repository	Central storage for models, requirements and metadata.
Diagrams	Create/edit/validate DFD, ERD and UML.
Code skeleton	Generate initial classes/interfaces/scripts where supported.
Traceability	Connect requirements to models, code and tests.
Consistency	Shared definitions reduce inconsistency.
Documentation	Generate reports from repository data.

14. Explain Structured English constructs and transform a multi-condition rule.

Construct	Meaning	Example
Sequence	Statements execute in order.	Read amount; Calculate tax; Display.
Selection	Choose based on condition.	IF member THEN discount ELSE no discount.
Iteration	Repeat while/until condition.	WHILE stock > 0 DO process item.

```
READ purchase amount AND membership status
IF purchase amount > 500 THEN
IF member THEN APPLY 15% DISCOUNT ELSE APPLY 10% DISCOUNT ENDIF
ELSE
IF member THEN APPLY 5% DISCOUNT ELSE APPLY NO DISCOUNT ENDIF
ENDIF
DISPLAY final amount
```

15. Discuss formal Economic Feasibility. Explain NPV and Break-Even Analysis.

- 1. Identify initial and recurring costs.
- 2. Estimate benefits/savings.
- 3. Select analysis period and discount rate.
- 4. Calculate period-by-period net cash flows.
- 5. Calculate NPV.
- 6. Calculate break-even/payback.
- 7. Perform sensitivity analysis.

$$NPV = \sum [Net\ Cash\ Flow_t / (1+r)^t] - Initial\ Investment$$

Break-even occurs when cumulative benefits/savings recover cumulative costs. The result depends on assumptions, discount rate and analysis period.

16. Explain SQA and compare Code Inspections, Structured Walkthroughs and Technical Reviews.

Software Quality Assurance is the planned set of activities ensuring software processes/products conform to requirements and quality standards.

Seven-point difference: Inspection vs Walkthrough

Basis	Code Inspection	Structured Walkthrough
1. Formality	Highly formal.	Moderately structured.
2. Focus	Systematic code defects.	Understanding/feedback.
3. Roles	Defined roles.	Usually author-led.
4. Process	Checklists, preparation, records.	Presentation and discussion.
5. Artifact	Source code.	Requirements/design/code.
6. Goal	Early defect detection.	Improvement and shared understanding.
7. Output	Defect/inspection record.	Issues and action items.

Seven-point difference: Walkthrough vs Technical Review

Basis	Structured Walkthrough	Technical Review
1. Focus	Understanding and feedback.	Technical correctness/quality.
2. Leadership	Often author-led.	Technical reviewers.
3. Formality	Moderate.	Structured/formal.
4. Artifact	Many work products.	Requirements/design/code etc.
5. Goal	Find issues and improve understanding.	Evaluate technical quality.
6. Method	Presentation/discussion.	Review against criteria.
7. Output	Suggestions/action items.	Findings/decisions.

Section C – Long Answer Questions

17. COMPULSORY CASE STUDY – Automated Public Transit & Smart Ticketing

The source scenario includes RFID cards, mobile-banking top-up, tap-in/tap-out, GPS route transmission, dispatch monitoring, rerouting and management reports.

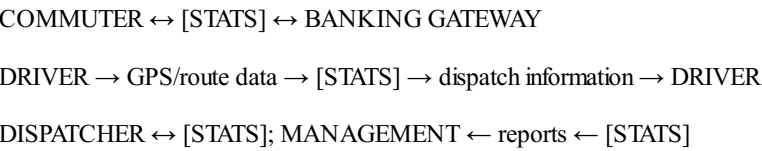
a. Stakeholders and requirements

Stakeholder	Interaction
-------------	-------------

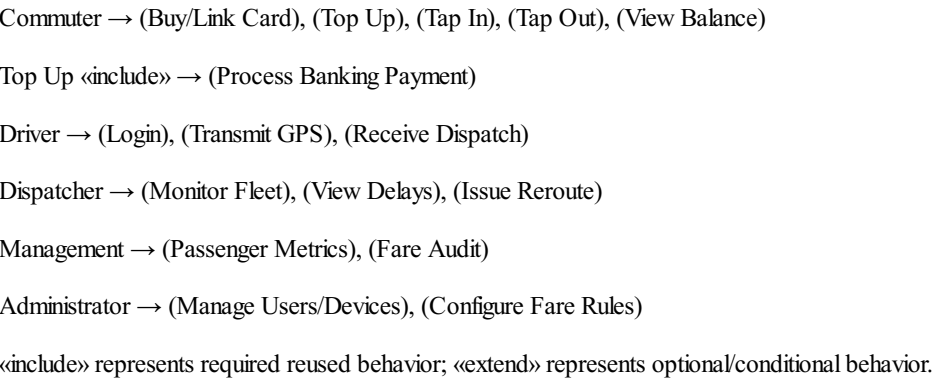
Commuter	Buy/top up cards, tap in/out, view balance.
Driver	Log in and transmit GPS/route data.
Dispatcher	Monitor routes, delays and rerouting.
Banking Gateway	Process top-up payments.
Management	Passenger metrics and revenue audit.
Administrator	Users, devices, configuration and security.

Functional: RFID card management; balance top-up; automated fare deduction. Non-functional: high availability; secure financial/passenger data.

b. Context Diagram



c. Use Case Diagram – text form



d. Installation strategy

Use a pilot plus phased rollout. Start at one route/station, test RFID reading, fare calculation, banking integration and recovery, then expand route-by-route. Temporary dual support for paper tickets can reduce passenger disruption until the new system is stable.

18. Critically evaluate Agile (Scrum/XP) versus Spiral.

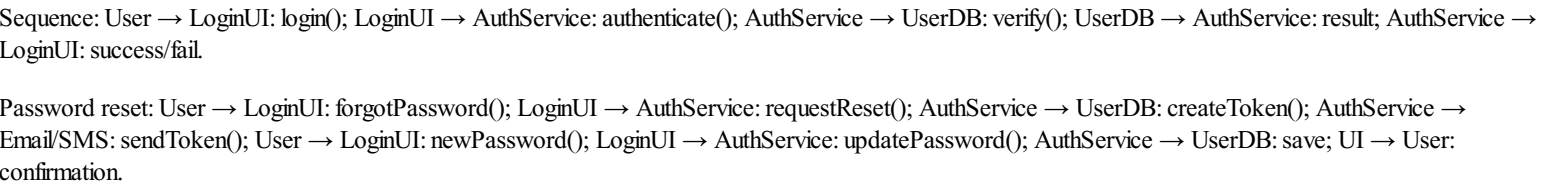
Seven-point difference: Agile vs Spiral

Basis	Agile	Spiral
1. Delivery	Short sprints/iterations.	Risk-driven cycles.
2. Client involvement	Frequent and continuous.	Review at cycle milestones.
3. Risk handling	Continuous through testing/feedback.	Explicit risk analysis each cycle.
4. Requirements	Evolve through backlog.	Evolve between cycles.
5. Documentation	Working software prioritized; necessary docs maintained.	More formal risk/design artifacts.
6. Team style	Cross-functional and adaptive.	Risk-driven planning/review.
7. Best fit	Changing requirements/fast feedback.	Large/high-risk systems.

Agile fits situations where requirements evolve and stakeholders can provide frequent feedback. Risk-driven models are useful when formal risk analysis is central. A hybrid can combine iterative delivery with strong governance.

19. Discuss Object Modeling with UML. Explain Sequence and Collaboration Diagrams and show authentication/password reset.

Object modeling identifies classes/objects, attributes, responsibilities and relationships. A Sequence Diagram emphasizes time order; a Communication/Collaboration Diagram emphasizes links among objects and numbered messages.



20. Explain UI and Output Design. Discuss human factors, layout, validation, errors and reports.

UI and Output Design aims to make interaction clear, efficient, accurate, accessible and appropriate to user tasks. Output design presents information in useful forms for operations and management.

- Use consistent labels, terminology and navigation.
- Group related fields and create clear visual hierarchy.
- Use readable layouts and accessible controls.
- Minimize memory load and unnecessary entry.
- Validate presence, type, format, range and cross-field consistency.
- Use clear error messages identifying the problem and correction.
- Use receipts, confirmations, invoices and reports appropriate to the user's task.

Management reports summarize trends, exceptions, workload, revenue and performance. Accurate labels, dates, summaries and clear presentation help managers make informed decisions.

QUICK REVISION SHEET

Topic	Key point
SDLC	Planning → Analysis → Design → Development → Testing → Implementation → Maintenance.
DFD	External Entity + Process + Data Flow + Data Store.
DFD errors	Black Hole=input/no output; Miracle=output/no input; Gray Hole=unjustified output.
ERD	Entities + Attributes + Keys + Relationships + Cardinality + Modality.
Decision Table	Condition stubs/entries + Action stubs/entries.
Feasibility	Technical + Operational + Economic + Schedule.
Design	High cohesion + loose coupling.
UML	Use Case + Class + Sequence + State Machine.
ROI	$(\text{Net Benefit} / \text{Investment Cost}) \times 100\%$.
NPV	$\Sigma[\text{Net Cash Flow}/(1+r)^t] - \text{Initial Investment}$.
Testing	Unit → Integration → System → Acceptance.
Maintenance	Corrective + Adaptive + Perfective + Preventive.
Conversion	Direct + Parallel + Pilot + Phased.
Normalization	1NF atomic; 2NF removes partial dependency; 3NF removes transitive dependency.

This document covers all questions appearing across the four model sets in the uploaded SAD paper and preserves the source's section structure and mark allocation. filecite turn1file0 L300-L371 